

第 4 组设计文档

模块：工具注册 + OS Skills（工具服务层） 对应 UF0² 组件：Tools | 所属层次：工具服务层
| OS 原理关联：持久性（文件系统）

- 课程：操作系统设计 • Linux 桌面 Agentic OS
- 第 4 组：工具服务层
- 版本：v3.2（2026-09-11：危险指令硬拦截扩展为任意授权档位+coordinator 关闭均生效，558 测试全绿）

一、架构分析

1.1 第 4 组在系统架构中的位置

整个 Agentic OS 采用分层 + 审计的数据流（第 1→2→3→4 组，第 5 组协调）：



1.2 设计目标

目标	度量
统一性	上层只面对一份工具清单 + 一个调用入口 + 一种返回结构
可审计性	每次调用可统计（次数/结果），操作可回溯
安全性	危险/外发操作可被签名与风险分级约束
可联调性	提供与正式版同构的 Mock，其它组可离线开发
可移植性	Windows / Linux 双平台同一套代码

1.3 关键设计取舍

- 用系统调用代替脆弱点坐标：凡是 OS 能确定完成的（复制/移动/搜索/磁盘），一律工具化，不依赖 GUI 点击——这是第 4 组对“GUI 自动化可靠性”的根本态度；
- Schema 由代码推断：工具参数不透写两份（代码+Schema），而是由函数签名 + docstring 自动生成 JSON Schema，避免漂移；
- 结果统一归一：`{success, result, error?}` 让上层无感知差异；
- 工具与技能分层：工具=原子操作，技能=可编排的组合操作。

二、接口契约

2.1 输出接口（第 4 组 → 上层执行层）

按设计书第 4 组契约，统一为 `tool_result_json`：

```
{
  "success": true,           // 或 false
  "result": "...",          // 成功时的结果（字符串/结构化对象）
  "error": "..."           // 仅 success=false 时出现
}
```

本组在契约基础上的工程扩展（保证可审计）：

```
{
  "success": true,
  "result": "...",
  "tool": "organize_downloads", // 调用的工具名
  "elapsed_ms": 42,             // 耗时
  "ts": "2026-09-09T09:00:00"   // 时间戳（供审计/RAG）
}
```

2.2 输入接口（上层 → 第 4 组）

接口	参数	返回
<code>list_available_tools()</code>	—	工具清单[{name, description, parameters(Schema)}]
<code>list_skills()</code>	—	技能清单
<code>call(name, params)</code>	工具名 + 结构化参数	tool_result_json
<code>call_skill(name, params)</code>	技能名 + 结构化参数	tool_result_json
<code>get_stats()</code>	—	调用统计（总次数/各工具次数/最近结果）
<code>get_tool_signature(name)</code>	工具名	该工具 SHA256 签名
<code>register(name, func)</code>	工具名 + 实现	注册成功并自动签名

2.3 与其它组的对接

对端组	对接物	说明
第 3 组（执行层）	<code>call(name, params)</code> → tool_result_json	第 3 组在选择“GUI 或工具”时，确定性系统操作走第 4 组
第 2 组（任务规划）	工具清单 / 技能清单	规划阶段即知有哪些能力可用
第 5 组（协调/安全/RAG）	调用统计 / 审计时间戳 / 签名	审计与执行轨迹入库
全体（联调）	<code>MockToolRegistry</code> （61 工具同构）	无真实系统时离线复现

三、模块设计

3.1 ToolRegistry（core/tool_registry.py，63 工具）

职责：统一注册、统一调用、Schema 生成、调用统计、工具签名。

核心结构：

- tools: dict[name -> func]

正式实现

- _tool_signatures: dict[name -> sha256]

注册时自动签名

- register(name, func, aliases)

可注册别名（move_file≡move_path）

- call(name, params)

统一入口；try/except 归一错误

- list_available_tools()

导出含描述/Schema 的清单（供 LLM）

- get_stats()

调用统计

- get_tool_signature / list_tool_signatures / 完整性校验

功能域（61 工具归类）：

域	代表工具
文件	copy/move/delete/rename、create_file、checksum、compare_files
文件夹	create/delete/list_directory、sort_directory、index_files、manage_archive
文本	read_text_file、write_text_file、edit_file
批量/查找	batch_copy/move、find_large/old/empty_files、search_files、search_in_files
磁盘	disk_usage、directory_size
图片	get_image_metadata、rotate_image
系统	current_directory、run_command（沙箱+超时）
网络/浏览器	browser_search、browser_extract、send_email、read_image
应用/屏幕	app_open、app_send_message、read_qq_chat、screen_inspect、activate_window

浏览器视觉定位（新增 2 工具，11 个浏览器工具，对齐 docs/miniyu 浏览器与桌面视觉定位增强方案（DOM-SoM-坐标混合架构）.md）：

- 结构化优先——browser_snapshot / browser_find：默认 browser_snapshot 拿带稳定 ref 的元素清单直接操作；大页面用 browser_find（按目标文字/名称/角色局部搜索）返回可点击 ref，不取整页结构省 token。
- 视觉辅助——browser_inspect（SoM）：截“交互元素带编号覆盖层”图，Agent 看编号 som:N 用 browser_click/browser_type 操作；编号仅是视觉标签，真正执行落到当前 DOM 的稳定 ref（core/uitarget.py 统一 UITarget: ref / 视觉编号 / bbox/center_css）。
- 坐标兜底 + 会话失效：只有 DOM 与 SoM 都定位不了（Canvas/地图/白板）才允许坐标点击；单一 InspectSession + URL/DOM 指纹做失效判定，页面变化旧 som:N 抛 SoMStaleError 逼重新 inspect。
- 统一寻址 + 客观校验：browser_click/browser_type 统一 target 兼容 ref / som:N / CSS 选择器三套；坐标存 CSS 视口像素、按截图 DPR 换算设备像素落点；动作后返回客观 url_changed / page_changed。

浏览器会话隔离与窗口可见性（2026-09-12 新增）： - 会话硬门控：browser_launch/navigate 成功后置 _browser_active，浏览器任务期间拦截 click_at/send_text/send_hotkey/activate_window 桌面全局工具（读操作放行），杜绝“网页任务误切桌面客户端”（豆包失败案例主因）；browser_close 解除。 - 窗口前置：browser_launch/navigate 后自动 SetForegroundWindow、并提供 browser_bring_to_front，让用户可见每次浏览器操作；尽力而为（Wayland 需 Xorg）。 - SPA 恢复 + 幂等：browser_refresh（Page.reload）作为 wait→refresh→re-snapshot 恢复链路；browser_launch 增存活探测，同档案不二次 spawn 抢 9222 端口。 - 持久登录档案：config.yaml1 broker.user_data_dir 指向独立专用档案路径，登录态跨会话保留、browser_close 不清除，实现“一次登录、长期复用”。

过程可观测（Token/想法/操作/时间，2026-09-12 新增）： - 每次运行生成一份 `run_log*.jsonl`（会话产物目录），逐行流水线覆盖两类事件： - `kind=llm`：时间戳、步号、该步模型的想法（`reasoning`），prompt/completion/total token、携带图片数、消息数、字符数、累计消耗； - `kind=action`：时间戳、步号、工具名、参数（关键字段脱敏）、成功与否、耗时、结果摘要。 - 最终回复末尾自动附“本轮/本对话/预算”三栏 token 概览（无真实消耗时不显示）；新增 `token_usage` 只读工具可查分步明细；供后续“截图去噪 + Observation 生命周期”优化取数。 - Observation 生命周期（P1，2026-09-11）：发往 LLM 的历史最多保留最近 2 张观测截图（`_MAX_HISTORY_IMAGES`），更旧的图片从当次载荷剥离、文字保留；对话持久记录与产物目录存全量原图。配套“结构化优于截图”：成功的 `snapshot/find/type/wait/read` 不再自动截图，仅 `launch/navigate`、`click` 引起页面/URL 变化、以及失败诊断时才截（桌面全局操作保留保底截图）；System Prompt 第 14 条要求读网页文字/状态一律走 `browser_read_text/snapshot`。实测单图=2466 token 且随图片数严格线性累加、末轮图片占输入约 50%，此策略为化解“旧图重发”主因的第一刀。

3.2 SkillLibrary (core/skill_library.py, 26 技能)

职责：把多个原子工具 + 业务逻辑编排成“一步到位”的组合技能，对应设计书“OS Skills 库”。

类别	技能
文件整理	<code>organize_downloads</code> 、 <code>smart_organize</code> 、 <code>cleanup_temp</code> 、 <code>cleanup_by_type</code> 、 <code>batch_archive</code> 、 <code>summarize_files</code> 、 <code>deduplicate_files</code>
查找/索引	<code>search_file</code> 、 <code>find_large/recent_files</code> 、 <code>rebuild_index</code> 、 <code>query_index</code> 、 <code>duplicate_finder</code>
回收/备份	<code>trash_file</code> 、 <code>restore_file</code> 、 <code>empty_trash</code> 、 <code>safe_delete</code> 、 <code>backup_file</code> 、 <code>restore_backup</code>
系统/应用	<code>system_info</code> 、 <code>app_open</code>
通信/联网	<code>app_send_message</code> 、 <code>read_qq_chat</code> 、 <code>send_email</code> 、 <code>browser_search</code> 、 <code>browser_extract</code>

技能示例（`organize_downloads`）——正是设计书演示场景 6「整理下载目录」：

```
输入: organize_downloads("~/Downloads")
逻辑: 扫描目录所有文件 → 按扩展名映射类别(文档/图片/视频/音频/压缩包/...) →
      逐类创建子目录 → 移动文件 → 返回"整理完成"+ 分类统计
```

3.3 MockToolRegistry (mock/mock_tools.py, 61 工具全量 Mock)

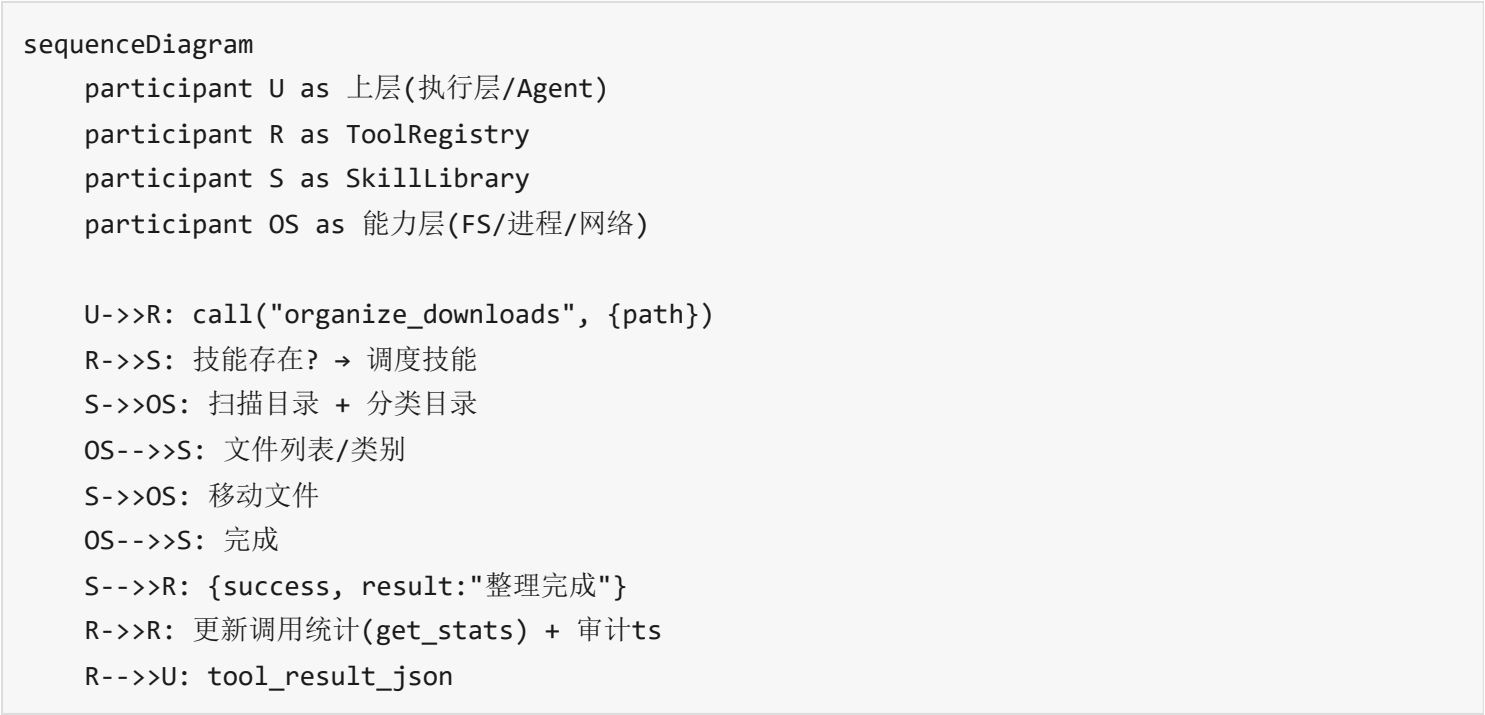
- 与正式版同构（同名、同参、同返回 `todo_result_json`），实现零负担切换；
- 内存实现（不改磁盘），保证跨组联调与测试的安全与可复现；
- 提供 `MockToolRegistry`，`MockSkillLibrary`，`MockOSServiceAPI` 三个类，对应各层接口。

3.4 安全与审计（与第 5 组协同）

- 危险/外发操作（删除、清空回收站、QQ 发消息、发邮件）由上层安全确认；第 4 组在工具实现侧仍保留白名单与风险标注；
- 每次 `call` 记录时间戳/耗时/结果到统计与审计，供第 5 组 RAG 与审计日志使用。

四、流程图

4.1 一次工具调用时序（call）



4.2 数据流与审计



五、可行性分析

维度	分析	结论
技术可行	核心基于 Python 标准库 <code>shutil</code> / <code>subprocess</code> / <code>pathlib</code> / <code>psutil</code> ，全跨平台、零外部重依赖	高
接口可行	统一 <code>tool_result_json</code> 已与设计书契约、MCP <code>CallToolResult</code> 对齐	高
联调可行	提供 61 工具同构 Mock，其它组无需真实系统即可联调	高
测试可行	583 项测试全绿，含统计/签名/异常/恢复/协调层/危险指令硬拦截/浏览器视觉定位等	高
跨平台可行	Windows 实测通过；路径/编码/临时目录均已做双平台适配（Ubuntu 为验收目标）	高

六、风险评估与对策

风险	影响	对策
危险操作误执行（删除/格式化）	用户数据丢失	删除类/外发类工具标注高风险，配合第 5 组安全确认；测试只在临时/安全目录跑
工具参数由 LLM 生成可能不合法	运行时异常	<code>call()</code> 统一 try/except 归一为 <code>{success:false,error}</code> ；Schema 校验参数
跨平台行为差异（Win/Linux 路径）	Linux 验收失败	全 <code>pathlib</code> + 双平台适配（ <code>app_controller</code> 分实现、CDP 跨平台）；已做 Linux 可移植性专项
工具输出超长占用 token	请求慢/贵	超长结果自动截断（默认前 4000 字符）+ 可针对性再读（09-08 落地）
个人 key 泄漏随代码分发	资损/安全	<code>config.yaml</code> 个人敏感字段靠 <code>.gitignore</code> + <code>skip-worktree</code> 不入库，分发的是空模板 <code>config.yaml.example</code> （09-09 落地）
其它组接口变更	集成失败	严守接口契约 + ADR 记录决策（ <code>docs/adr/*</code> ）追踪演进

配套文档：`docs/第4组调研报告.md`、`docs/第4组接口文档.md`、`docs/第4组单元测试报告.md`、`docs/第4组联调与集成报告.md`、`docs/第4组交付说明.md`、`docs/adr/*`。